

NOMBRES: Sebastián Giraldo Franco y Juan Jose Jaramillo Mora.

BIG DATA · TRABAJO 3 (TERCERA NOTA)

Pipeline de streaming en tiempo real

Detección de patrones en vivo con Kafka + Spark Structured Streaming

Prof. Alejandro Mery Agudelo · En parejas

De qué se trata

Construirán un pipeline de streaming completo que consume un flujo de eventos desde Kafka y detecta patrones en tiempo real usando Spark Structured Streaming. Es el paso final de la unidad: aplicar todo lo visto (Kafka, parseo de eventos, ventanas de tiempo, estado y reglas) a un problema de negocio real, de principio a fin.

Cómo funciona este trabajo

- Elijan UNO de los tres casos de negocio descritos abajo. Los tres tienen nivel de dificultad equivalente; elijan el dominio que más les interese.
- El profe entrega el PRODUCTOR de Kafka de su caso (genera el flujo de eventos con los patrones ya sembrados). Ustedes construyen TODO lo demás.
- Trabajo en PAREJAS. Esta es la tercera nota formal del curso.

Qué les da el profe / qué construyen ustedes

Les entrega el profe: el script productor del caso (p. ej. productor_flota.py), que envía los eventos a Kafka. No lo modifiquen; es su fuente de datos.

Construyen ustedes:

- El montaje de Kafka en Docker y la creación del topic (como en la Clase 12).
- El detector con Spark Structured Streaming: leer de Kafka, parsear el JSON, e implementar las 3 reglas de detección de su caso.
- El informe de negocio: interpretar lo que detectan, no solo producir alertas.
- La evidencia de que el pipeline corrió en vivo (capturas del detector alertando).

Requisitos técnicos (comunes a los tres casos)

- Kafka levantado en Docker (reutilicen el entorno de la Clase 12).
- Spark leyendo de Kafka con el conector spark-sql-kafka (se descarga al arrancar).
- Spark y Kafka en la MISMA máquina (Colab en la nube no ve su Kafka local).

Nota técnica (aplica sobre todo al Caso C, y a la regla 2 del Caso A)

foreachBatch en caso de problemas con las windows functions ;)

CASO A

Monitoreo de una flota de vehículos

Topic de Kafka: flota

Una empresa de logística monitorea su flota en tiempo real. Cada vehículo emite telemetría continuamente: velocidad, nivel de combustible y posición. Deben construir un tablero de alertas que detecte problemas al instante.

Cada evento trae: id_vehiculo, velocidad, combustible, lat, lon.

Reglas a implementar (las 3)

1. EXCESO DE VELOCIDAD SOSTENIDO: alertar cuando un vehículo supera los 100 km/h en 3 o más lecturas dentro de una ventana de tiempo (no un pico aislado, sino sostenido).
2. CAÍDA ANÓMALA DE COMBUSTIBLE: alertar cuando el combustible de un vehículo baja mucho más rápido de lo normal entre lecturas (posible robo o fuga). Pista: necesitan comparar cada lectura con la anterior del mismo vehículo.
3. VEHÍCULO EN SILENCIO: detectar vehículos que venían emitiendo y dejaron de hacerlo (posible falla o accidente). Pista: rastreen la última emisión de cada vehículo.

Preguntas de negocio: ¿qué vehículo dio más alertas? ¿el silencio se detecta rápido o tarde? ¿qué harían en producción con cada tipo de alerta?

El detector con Spark Structured Streaming identificó los tres patrones sembrados en la flota (exceso de velocidad de V03, caída de combustible de V05 y silencio de V07) con 56 alertas y sin falsos positivos.

1. Caso elegido

Elegimos el Caso A (flota de vehículos) porque su productor siembra tres patrones conocidos, lo que permite comprobar si el detector acierta y no solo mostrar que "emite alertas". Cada evento trae id_vehiculo, velocidad, combustible, lat y lon, y el productor emite una lectura por segundo de cada uno de los 8 vehículos (V01 a V08).

Vehículo	Patrón sembrado por el productor	Regla que debería dispararse
V03	Velocidad de 115 a 140 km/h durante 16 lecturas seguidas, en ciclos de 60 s	Regla 1: exceso de velocidad sostenido
V05	Consumo de 2,5 a 4,0 puntos por lectura (lo normal es 0,05 a 0,2), en ciclos de 80 s	Regla 2: caída anómala de combustible
V07	Deja de emitir en un instante aleatorio entre los segundos 60 y 90	Regla 3: vehículo en silencio

El código (detector.py, resumen.py), el registro de alertas (alertas_log.csv) y las capturas están en el repositorio de GitHub del trabajo.

2. Arquitectura del pipeline

El productor, entregado por el profesor, envía eventos JSON al topic flota de Kafka; el detector de Spark los lee en micro-lotes de 2 s, aplica las tres reglas y registra cada alerta.



3. Cómo funciona el detector

El detector (detector.py) lee el topic flota con el conector spark-sql-kafka, parsea el JSON con un esquema y usa como tiempo del evento el timestamp que Kafka asigna a cada mensaje, porque el productor no incluye hora. Procesa micro-lotes de 2 s con `spark.sql.shuffle.partitions = 4` (el valor por defecto, 200, es excesivo para este volumen) y guarda cada alerta en consola y en `alertas_log.csv`.

Parámetro	Valor	Justificación
Umbral de velocidad	100 km/h	Límite a partir del cual hay exceso
Lecturas mínimas	3	Distingue un exceso sostenido de un pico aislado
Ventana	10 s	El productor emite 1 lectura por segundo por vehículo: ~10 lecturas por ventana
Watermark	20 s	Tolerancia a datos tardíos; permite a Spark liberar estado
Caída de combustible	1,0 puntos	Entre el consumo normal (hasta 0,3 con redondeo) y el anómalo (2,5 a 4,0)
Silencio	10 s sin emitir	Margen para no confundir un retraso breve con una falla

Regla 1, exceso de velocidad sostenido. Se filtran las lecturas con velocidad mayor a 100, se agrupan por ventana de 10 s y vehículo con `window()` y `withWatermark()`, y se alerta cuando el conteo llega a 3. Usa

outputMode("update") y un conjunto de ventanas ya alertadas para no repetir la alerta cada vez que el conteo crece.

Regla 2, caída anómala de combustible. Spark no permite comparar cada fila con la anterior (lag) sobre un stream, así que cada micro-lote se procesa con foreachBatch, ordenado por offset, y un diccionario guarda el último nivel de combustible de cada vehículo. Se alerta si la caída entre dos lecturas consecutivas es de 1,0 o más; las subidas (el productor rellena el tanque a 100) se ignoran.

Regla 3, vehículo en silencio. En el mismo foreachBatch se guarda el timestamp del último evento de cada vehículo y se compara con el "reloj del stream" (el timestamp más reciente visto en cualquier vehículo). Si la diferencia supera 10 s se alerta una sola vez, y si el vehículo vuelve a emitir se limpia su estado.

4. Resultados por regla

En la corrida de validación (unos 3 min 40 s de telemetría, 220 ticks del productor) el detector registró 56 alertas, todas sobre los tres vehículos con anomalía sembrada y ninguna sobre los otros cinco.

Regla	Vehículo	Alertas	Episodios	Primera alerta
1. Exceso de velocidad sostenido	V03	8	4	21:59:30
2. Caída anómala de combustible	V05	47	3	21:59:48
3. Vehículo en silencio	V07	1	1	22:00:37
Sin anomalía sembrada	V01, V02, V04, V06, V08	0	0	–

R/ Regla 1 (V03). Se detectaron 4 episodios, uno cada 60 s (21:59:30, 22:00:30, 22:01:30 y 22:02:30), igual que el ciclo del productor. Cada racha de 16 lecturas rápidas cae sobre dos ventanas consecutivas de 10 s, por eso son 8 alertas, dos por episodio, con velocidades máximas de 130 a 140 km/h.

```
Problemas 61 Salida Terminal Puertos Consola de depuración powershell +
rigger interval is 2000 milliseconds, but spent 2176 milliseconds

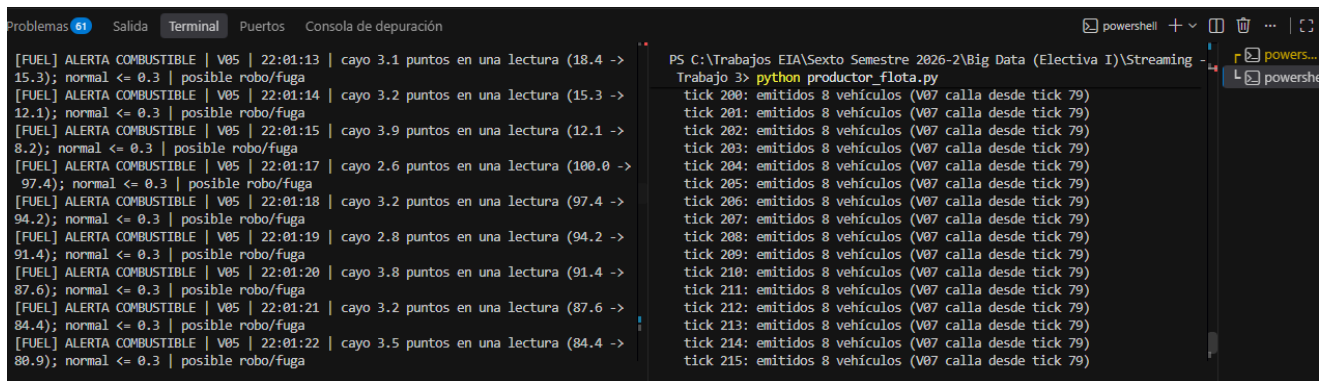
[VEL] ALERTA VELOCIDAD | V03 | 21:59:30 | 4 lecturas > 100 km/h en ventana 21:59:30
-21:59:40 (max 134 km/h)
[VEL] ALERTA VELOCIDAD | V03 | 21:59:40 | 4 lecturas > 100 km/h en ventana 21:59:40
-21:59:50 (max 137 km/h)

[FUEL] ALERTA COMBUSTIBLE | V05 | 21:59:48 | cayo 3.9 puntos en una lectura (95.4 ->
91.5); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 21:59:49 | cayo 3.6 puntos en una lectura (91.5 ->
87.9); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 21:59:50 | cayo 3.3 puntos en una lectura (87.9 ->
84.6); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 21:59:51 | cayo 3.7 puntos en una lectura (84.6 ->
80.9); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 21:59:52 | cayo 3.8 puntos en una lectura (80.9 ->
77.1); normal <= 0.3 | posible robo/fuga

PS C:\Trabajos EIA\Sexto Semestre 2026-2\Big Data (Electiva I)\Streamin
Trabajo 3> python productor_flota.py
tick 29: emitidos 8 vehículos (V07 calla desde tick 79)
tick 30: emitidos 8 vehículos (V07 calla desde tick 79)
tick 31: emitidos 8 vehículos (V07 calla desde tick 79)
tick 32: emitidos 8 vehículos (V07 calla desde tick 79)
tick 33: emitidos 8 vehículos (V07 calla desde tick 79)
tick 34: emitidos 8 vehículos (V07 calla desde tick 79)
tick 35: emitidos 8 vehículos (V07 calla desde tick 79)
tick 36: emitidos 8 vehículos (V07 calla desde tick 79)
tick 37: emitidos 8 vehículos (V07 calla desde tick 79)
tick 38: emitidos 8 vehículos (V07 calla desde tick 79)
tick 39: emitidos 8 vehículos (V07 calla desde tick 79)
tick 40: emitidos 8 vehículos (V07 calla desde tick 79)
tick 41: emitidos 8 vehículos (V07 calla desde tick 79)
tick 42: emitidos 8 vehículos (V07 calla desde tick 79)
tick 43: emitidos 8 vehículos (V07 calla desde tick 79)
tick 44: emitidos 8 vehículos (V07 calla desde tick 79)
```

R/ Regla 2 (V05). Se detectaron 3 episodios, uno cada 80 s (21:59:48, 22:01:08 y 22:02:28), con caídas de 2,5 a 4,0 puntos por lectura frente a un consumo normal de hasta 0,3. Las 47 alertas suman 16 + 15 + 16 lecturas:

en el segundo episodio el nivel bajó de 5 y el productor rellenó el tanque a 100, y esa lectura (8,2 a 100,0) sube en vez de bajar, así que no genera alerta.

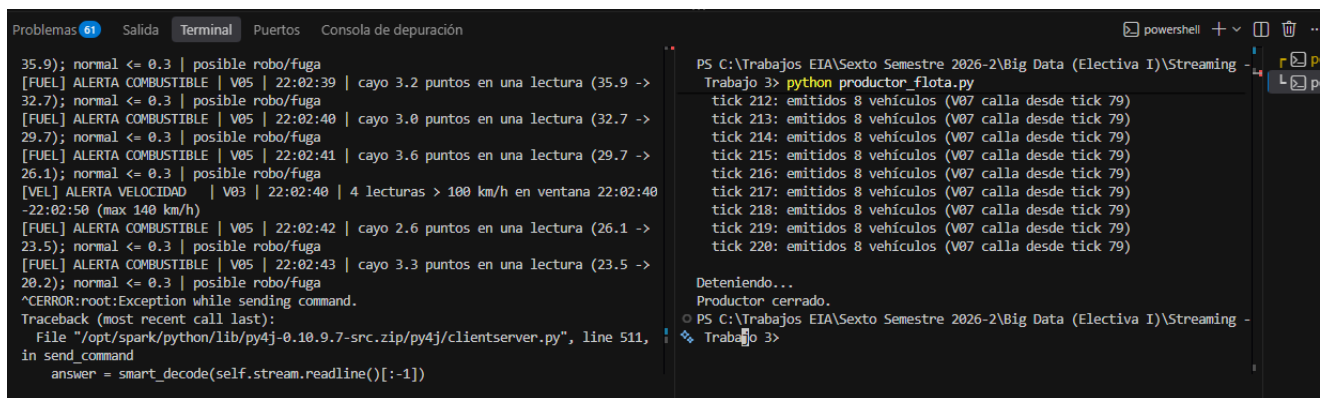


```
Problemas 61 Salida Terminal Puertos Consola de depuración

[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:13 | cayo 3.1 puntos en una lectura (18.4 -> 15.3); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:14 | cayo 3.2 puntos en una lectura (15.3 -> 12.1); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:15 | cayo 3.9 puntos en una lectura (12.1 -> 8.2); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:17 | cayo 2.6 puntos en una lectura (100.0 -> 97.4); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:18 | cayo 3.2 puntos en una lectura (97.4 -> 94.2); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:19 | cayo 2.8 puntos en una lectura (94.2 -> 91.4); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:20 | cayo 3.8 puntos en una lectura (91.4 -> 87.6); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:21 | cayo 3.2 puntos en una lectura (87.6 -> 84.4); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:01:22 | cayo 3.5 puntos en una lectura (84.4 -> 80.9); normal <= 0.3 | posible robo/fuga

PS C:\Trabajos EIA\Sexto Semestre 2026-2\Big Data (Electiva I)\Streaming -
Trabajo 3> python productor_flota.py
tick 200: emitidos 8 vehículos (V07 calla desde tick 79)
tick 201: emitidos 8 vehículos (V07 calla desde tick 79)
tick 202: emitidos 8 vehículos (V07 calla desde tick 79)
tick 203: emitidos 8 vehículos (V07 calla desde tick 79)
tick 204: emitidos 8 vehículos (V07 calla desde tick 79)
tick 205: emitidos 8 vehículos (V07 calla desde tick 79)
tick 206: emitidos 8 vehículos (V07 calla desde tick 79)
tick 207: emitidos 8 vehículos (V07 calla desde tick 79)
tick 208: emitidos 8 vehículos (V07 calla desde tick 79)
tick 209: emitidos 8 vehículos (V07 calla desde tick 79)
tick 210: emitidos 8 vehículos (V07 calla desde tick 79)
tick 211: emitidos 8 vehículos (V07 calla desde tick 79)
tick 212: emitidos 8 vehículos (V07 calla desde tick 79)
tick 213: emitidos 8 vehículos (V07 calla desde tick 79)
tick 214: emitidos 8 vehículos (V07 calla desde tick 79)
tick 215: emitidos 8 vehículos (V07 calla desde tick 79)
```

R/ Regla 3 (V07). Se generó una sola alerta: el último dato de V07 llegó a las 22:00:26 y el detector lo declaró en silencio a las 22:00:37, tras 11 s sin emitir. Coincide con el instante en que el productor lo silencia (tick 79).



```
Problemas 61 Salida Terminal Puertos Consola de depuración

35.9); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:02:39 | cayo 3.2 puntos en una lectura (35.9 -> 32.7); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:02:40 | cayo 3.0 puntos en una lectura (32.7 -> 29.7); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:02:41 | cayo 3.6 puntos en una lectura (29.7 -> 26.1); normal <= 0.3 | posible robo/fuga
[VEL] ALERTA VELOCIDAD | V03 | 22:02:40 | 4 lecturas > 100 km/h en ventana 22:02:40 -22:02:50 (max 140 km/h)
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:02:42 | cayo 2.6 puntos en una lectura (26.1 -> 23.5); normal <= 0.3 | posible robo/fuga
[FUEL] ALERTA COMBUSTIBLE | V05 | 22:02:43 | cayo 3.3 puntos en una lectura (23.5 -> 20.2); normal <= 0.3 | posible robo/fuga
^ERROR:root:Exception while sending command.
Traceback (most recent call last):
  File "/opt/spark/python/lib/py4j-0.10.9.7-src.zip/py4j/clientserver.py", line 511, in send_command
    answer = smart_decode(self.stream.readline()[:-1])

PS C:\Trabajos EIA\Sexto Semestre 2026-2\Big Data (Electiva I)\Streaming -
Trabajo 3> python productor_flota.py
tick 212: emitidos 8 vehículos (V07 calla desde tick 79)
tick 213: emitidos 8 vehículos (V07 calla desde tick 79)
tick 214: emitidos 8 vehículos (V07 calla desde tick 79)
tick 215: emitidos 8 vehículos (V07 calla desde tick 79)
tick 216: emitidos 8 vehículos (V07 calla desde tick 79)
tick 217: emitidos 8 vehículos (V07 calla desde tick 79)
tick 218: emitidos 8 vehículos (V07 calla desde tick 79)
tick 219: emitidos 8 vehículos (V07 calla desde tick 79)
tick 220: emitidos 8 vehículos (V07 calla desde tick 79)

Deteniendo...
Productor cerrado.
PS C:\Trabajos EIA\Sexto Semestre 2026-2\Big Data (Electiva I)\Streaming -
Trabajo 3>
```

Resumen:

```
Problemas 61 Salida Terminal Puertos Consola de depuración

python resumen.py

TOTAL DE ALERTAS: 56

Por regla:
  VELOCIDAD      8
  COMBUSTIBLE    47
  SILENCIO       1

Por vehiculo (ordenado):
  Vehiculo  VELOCIDAD  COMBUSTIBLE  SILENCIO  TOTAL
  V05       0         47         0         47
  V03       8         0         0         8
  V07       0         0         1         1

Vehiculo con mas alertas: V05

Primera alerta de cada regla:
```

```
Problemas 61 Salida Terminal Puertos Consola de depuración

  Vehiculo  VELOCIDAD  COMBUSTIBLE  SILENCIO  TOTAL
  V05       0         47         0         47
  V03       8         0         0         8
  V07       0         0         1         1

Vehiculo con mas alertas: V05

Primera alerta de cada regla:
  VELOCIDAD    2026-10-05 21:59:30 V03  4 lecturas > 100 km/h en ventana 21:59:30-21:59:40 (max 134 km/h)
  COMBUSTIBLE  2026-10-05 21:59:48 V05  cayo 3.9 puntos en una lectura (95.4 -> 91.5); normal <= 0.3 | posible robo/fuga
  SILENCIO     2026-10-05 22:00:37 V07  sin emitir hace 11 s (ultimo dato 22:00:26) | posible falla/accidente
PS C:\Trabajos EIA\Sexto Semestre 2026-2\Big Data (Electiva I)\Streaming - Trabajo 3>
█
```

Docker:

☐	▼	●	streaming-traba	-	-	1.98%	22 minutes ago				
☐		●	spark-flota	10a454a38c9a	apache/spar	0%	22 minutes ago				
☐		●	kafka-flota	71cfd347aa73	apache/kafk 9092:9092	1.98%	22 minutes ago				

Showing 18 items

RAM 7.03 GB CPU 1.17% Disk: 56.09 GB used (limit 1006.85 GB)

Terminal v4.94.0

5. Preguntas de negocio

¿Qué vehículo dio más alertas?

R/ V05, con 47 de las 56 alertas (84 %), seguido de V03 (8) y V07 (1). El conteo, sin embargo, no es comparable entre reglas: la de combustible alerta en cada lectura anómala (una por segundo durante todo el episodio), la de velocidad alerta una vez por ventana y la de silencio una sola vez. Contado por episodios el orden cambia: V03 (4), V05 (3) y V07 (1). Y por urgencia operativa, la única alerta de V07 es la que exige la respuesta más rápida.

¿El silencio se detecta rápido o tarde?

R/ Con 11 s de latencia: es rápido para una flota logística, pero no instantáneo, porque un vehículo solo puede declararse en silencio cuando pasa el umbral (10 s) sin datos, más el intervalo del micro-lote (2 s). Es un compromiso: un umbral menor detecta antes, pero confunde un retraso de red con una falla real.

¿Qué haríamos en producción con cada tipo de alerta?

R/

Alerta	Acción propuesta
Exceso de velocidad sostenido (V03)	Avisar al conductor y al despachador en el momento, y acumular el historial por conductor para programas de seguridad vial
Caída anómala de combustible (V05)	Contactar al conductor, verificar la ubicación del vehículo con lat y lon y, si no hay explicación, escalar como posible robo o fuga
Vehículo en silencio (V07)	Llamar al conductor y, si no responde, enviar apoyo a la última posición conocida

Antes de llevarlo a producción, el estado de las reglas 2 y 3 (hoy en la memoria del detector, que se pierde si este se reinicia) debería vivir en un almacén persistente, por ejemplo con `applyInPandasWithState` de Spark o con Redis.

6. Retos técnicos

Reto	Cómo lo resolvimos
docker compose up falló porque el contenedor kafka del laboratorio de la Clase 12 ya existía con ese nombre	Renombramos los contenedores a kafka-flota y spark-flota, y fijamos name: streaming-trabajo3 en el compose para que queden en un solo grupo. El Kafka del laboratorio se conserva, pero debe estar detenido: el productor usa localhost:9092 fijo
Spark y Kafka debían estar en la misma máquina, y Colab no ve el Kafka local	Spark corre en un contenedor de la misma red de Docker. Kafka tiene dos listeners: localhost:9092 para el productor en Windows y kafka:29092 para Spark
El conector spark-sql-kafka no viene incluido	Se descarga al arrancar con --packages; un volumen de Docker conserva la descarga para no repetirla
El laboratorio crea el topic con 3 particiones, y Kafka solo garantiza el orden dentro de una partición	Creamos flota con 1 partición: el offset da un orden total de llegada, necesario para comparar cada lectura con la anterior de su vehículo
Spark no permite lag sobre un stream, y las reglas 2 y 3 necesitan lecturas previas	Usamos foreachBatch, que entrega cada micro-lote como DataFrame normal, y guardamos el estado (último combustible y última emisión por vehículo) en el detector
Al detener el detector con Ctrl+C en pleno micro-lote, Py4J imprimió un traceback largo (InterruptedException) y no salió el resumen final	No se perdió información: cada alerta ya estaba en alertas_log.csv y el resumen se obtiene con resumen.py. Para una salida limpia, se detiene primero el productor y luego el detector
Aparecieron avisos Current batch is falling behind (entre 2,2 y 3,1 s frente al trigger de 2 s)	Solo ocurrieron al arrancar y al llegar los primeros datos; no volvieron a aparecer durante el resto de la corrida, así que no ajustamos el trigger

Qué y cómo entregar

1. El código de su detector (detector.py), con las 3 reglas de su caso implementadas y comentadas.
2. Un informe en PDF o .md (máx. 5 páginas) con: qué caso eligieron, cómo funciona su detector, los resultados de cada regla, las respuestas a las preguntas de negocio, y al menos una captura del detector alertando EN VIVO (evidencia de que el pipeline corrió).
3. Una breve sección de 'retos técnicos': qué les costó (montar Kafka, el conector, foreachBatch...) y cómo lo resolvieron.
4. Los nombres de la pareja y el caso elegido en la primera página.